Reinforcement Learning For Emergency Department Boarding

Eli Fried
PID: 730465100
elixf@ad.unc.edu
https://github.com/elixf7/RL-for-ED-Boarding

1 Introduction

Emergency department boarding is a common bottleneck in modern hospitals, which is defined as the complex process of allocating limited resources to incoming patients awaiting care. In a standard emergency department setting, patients are evaluated upon arrival and assigned an Emergency Severity Index (ESI) ranging from 1 to 5, with a lower number indicating higher medical acuity. The core operational challenge is balancing acuity values with accumulated wait times to assign resources such as trauma bays, main ED beds, and basic chairs to patients. Hospital administrators are tasked with ensuring that high-acuity patients receive immediate care while at the same time minimizing the overall wait time for all of the patients.

This is clearly a difficult problem for hospitals to approach since it requires balancing many variables at once, such as patient arrival rate, individual wait times, and the number of beds open. Some traditional approaches, such as "first-come, first-served" or "acuity first," often fall short since they do not take into account the dynamic environment of the emergency department. For instance, first-come, first-served might lower the overall wait times, but by assigning moderately ill patients to high-acuity beds, you could cause a bottleneck when critical trauma patients arrive. On the other hand, "acuity first" might never miss high-acuity arrivals, but wait times for lower-acuity patients might grow beyond what is reasonable. Overall, optimizing this environment requires a methodology that is capable of decision-making when some variables are uncertain and anticipating future states rather than purely reacting to the present one. This project aims to apply machine learning techniques to solve this problem.

To address the limitations of naive approaches, I propose modeling emergency department boarding as a Markov Decision Process and optimizing it with reinforcement learning (RL). Specifically, I implement a Deep Q-Network (DQN) to manage this environment. Through using a neural network to approximate the Q-value function, the model can evaluate the state space of the ED, including bed occupancies, waiting room distributions, and elapsed wait times. The output is the expected long-term reward for each possible action. The model is incentivized to maximize patient safety and throughput of patients with a reward function that penalizes excessive wait times by ESI level and routing errors, such as assigning high-acuity patients to a standard chair.

To ensure the RL agent learns a strategy that is applicable to real-world settings, the simulation environment is grounded in empirical data. The MIMIC-IV-ED dataset is composed of over 400,000 de-identified emergency department stays. From this, I can extract realistic ESI acuity distributions, hourly arrival rates, and length-of-stay estimates. Through using reinforcement learning and historical clinical data, this project aims to demonstrate whether an AI agent can manage the competing priorities of an emergency department better than standard baseline methods.

Preprint. Work in progress.

2 Related Work

Applying machine learning to healthcare operations such as emergency department bedding have seen growth as hospitals seek to use data to find better solutions to overcrowding. Many studies in this area rely on comprehensive clinical datasets that establish baseline ED visit patterns that better inform their models. Recent literature has analyzed the MIMIC-IV-ED dataset to extract realistic parameters. Wei et al. [1] analyzed over 425,000 ED cases in this dataset to establish distribution for patient throughput. Delos Reyes et al. [2] also utilized this dataset to extract precise arrival rates, ESI frequency distributions and acuity based length of stay medians. These takeaways are crucial for this project as they provide the exact statistical constants I needed to build a realistic simulation environment that is generalizable to actual hospital operations.

Reinforcement learning in particular has arisen as a highly effective method for patient scheduling. Lee and Lee [3] proposed a Deep Q-Network designed to schedule patients in an ED environment characterized by limited medical resources. Specifically, they focused on available medical staff and scaled their rewards by ESI level. I decided to adopt a similar methodology into this project and use a DQN with an acuity weighted reward function. I also adopted the double DQN approach which uses an online and target network combo to stabilize training targets and increase training efficiency. However, while Lee and Lee focused on matching patients to available medical personnel through a series of treatments, this project focuses on an action space composed of physical resources such as trauma bays, main ED beds, and chairs.

Furthermore, another paper by Li et al. [4] used a transformer based network to optimize allocation of scarce healthcare resources such as ventilators. Their model used a reward function to balance survival rates with demographic fairness penalties. My project uses some of these insights to build my own reward function that penalizes the misuse of rooms for the wrong acuity. However, their research worked on a day to day time frame for rationing resources while my environment addresses the continuous flow of patients into the waiting room. Rather than allocating a single life or death resource like a ventilator, my agent must optimize boarding where the penalty is not immediate death of the patient, but rather a weighted penalty depending on wait time and appropriate bed type. Through adapting the approaches of these related works my research aims to train an agent capable of managing the specific problem of emergency department boarding.

3 Methods

3.1 Simulation Environment

The central part of this project is a Markov Decision Process (MDP) that is implemented in Gymnasium which is a python library built for reinforcement learning environments. Since working with live hospital data is not feasible, I built a simulator which has parameters sourced from real data with the MIMIC-IV-ED dataset (Delos Reyes et al., 2024). The simulator runs in discrete time steps of 5 minutes giving a total of 288 steps per 24 hour episode. Another property of the environment is bed configuration. The ED is modeled as having three zones: Trauma Bays (2 beds), Main ED Beds (18 beds), and Chairs (8 beds) for a total of 54. The number of beds was halved from the original amount as a way to increase scarcity and demand on the system. Before this change too many beds were open making the problem trivial to solve by just assigning all patients to beds which is unrealistic in a real ED.

The MIMIC-IV-ED dataset was curated by Wei et al. [1] into a structured event log called MIMICEL which captured the end to end journey patients took through the ED. This was a vital resource for this project, table 1 below shows some of these summary statistics for the MIMICEL dataset.

Table 1: MIMICEL Dataset Summary Statistics (Wei et al., 2025)

Attribute	Value
Cases (stay_id)	425,028
Patients (subject_id)	205,466
Total events	7,568,824

Also from this dataset Delos Reyes et al. [2] extracted the ESI acuity distribution, hourly arrival rates, and median length of stay (LOS) by acuity level. These are three core statistical inputs into the simulation. Table 2 shows that ESI 3 make up the majority of cases accounting for over half of arrivals with ESI 2 the second most common at 33.0%.

Table 2: ESI Acuity Distribution from MIMIC-IV-ED (Delos Reyes et al., 2024)

ESI Level	Description	Proportion
1	Immediate, life-threatening	7.0%
2	High risk / emergent	33.0%
3	Urgent	54.0%
4	Less urgent	6.0%
5	Non-urgent	0.2%

I used these statistics to model new patients arriving according to a Poisson process which has a rate that varies by time of day following the counts from Delos Reyes et al. (2024):

$$\lambda_t = \begin{cases} 0.75 & \text{steps 0-31 (12 AM-8 AM, Night)} \\ 2.25 & \text{steps 32-63 (8 AM-4 PM, Day)} \\ 1.50 & \text{steps 64-95 (4 PM-12 AM, Evening)} \end{cases}$$

The steps are in 5 minute increments, and at each step the number of new arrivals is sampled from this process. Each arriving patient is also independently assigned an Emergency Severity Index (ESI) level from the distribution normalized to sum to 1:

$$P(\text{ESI} = k) \in \{0.070,\ 0.330,\ 0.540,\ 0.060,\ 0.002\}$$

These proportions also come from the MIMIC-IV-ED dataset as well where ESI 1 is a life threatening emergency and ESI 5 is non urgent.

Each patient is represented as a python class with fields for ID, ESI level, arrival time, and treatment duration. Treatment duration is sampled when the patient arrives rather than when they are given a bed so the model knows what to expect when making decisions. The duration is sampled from a log normal distribution:

$$d \sim \text{LogNormal}(\mu_k,\ \sigma)$$

I used a fixed standard deviation of $\sigma = 0.5$, and the mean is calculated so that the average duration matches the median length of stay in hours from the MIMIC-IV-ED dataset for each ESI level. The resulting averages are 20, 26, 23, 12, and 9 steps for each ESI level 1 to 5. Using the log normal distribution here is ideal since it is always positive, and produces a long tail since sometimes a small fraction of patients will need significantly longer treatment times.

3.2 State Space

The environment provides the agent with a 65-dimensional vector s_t at every step.

$$s_t = \big[\underbrace{p_1^{(1)}, p_1^{(2)}, p_1^{(3)},\ \ldots,\ p_{20}^{(1)}, p_{20}^{(2)}, p_{20}^{(3)}}_{\text{20 patient slots} \times 3},\ f_T,\ f_M,\ f_C,\ \sin(2\pi\tau),\ \cos(2\pi\tau)\big]$$

The first 60 indices encode up to 20 patient slots, each described by three normalized features which are ESI, wait time divided by the left without being seen threshold (LWBS, 4 hours), and treatment duration. All three attributes are critical so the agent is always aware of the patient who has been waiting the longest, discouraging patients from being stranded permanently in the wait room. The next three f_T, f_M, f_C are the number of available Trauma Bays, Main Beds, and Chairs. Finally, the last two are are the time of day represented as $\sin(2\pi\tau)$ and $\cos(2\pi\tau)$ since sine and cosine represent the cyclic nature of a day. This way the model knows that 11:45 PM and 12:00 AM are actually close together.

$$\tau = \frac{t \bmod 288}{288}, \qquad \sin(2\pi\tau),\ \cos(2\pi\tau)$$

3.3 Action Space

The agent chooses from 21 possible actions. Action 0 tells the agent to do nothing, actions 1-20 command the environment to admit a specific patient from the waiting room. These 20 actions are split up by ESI level and then sorted by wait time. For example, three slots are reserved for ESI 1 patients so the agent can choose to assign any of them to rooms. Seven slots are reserved for ESI 3 patients since they are more common. The overall distribution is: $\{3, 6, 7, 3, 1\}$ from ESI 1 to 5. I chose to implement the action space this way so that the model is not overwhelmed with every patient in the waiting room and instead sees those who have been waiting the longest. This method also guarantees that every ESI group is represented in the action space so long wait times do not hide high acuity patients.

Bed type assignment is also important for the model. ESI 1 and 2 should be assigned to a trauma bay if one is available and fall back on main bed otherwise. ESI 3 patients go to a main bed first with chair as a fallback. ESI 4 and 5 patients go to a chair first with a main bed as a fallback. These assignment rules make sure that an appropriate level of care is given to each patient and prevents the agent from misallocating resources.

3.4 Reward Function

The reward r_t at each step is calculated by combining a discharge bonus, admission incentive, per step acuity weighted wait penalties, clinical guideline penalties, and left without being seen (LWBS) penalties.

$$r_t = r_{\text{discharge}} + r_{\text{admit}} - r_{\text{wait}} - r_{\text{guideline}} - r_{\text{LWBS}}$$

The discharge bonus $r_{\text{discharge}}$ is every time a patient finishes treatment and frees up a bed, the agent receives a +20 for this to incentivize efficiency in discharging patients. If the patient is discharged from a suboptimal bed, for example and ESI 1 in a main bed, then that reward is reduced. A small admission reward r_{admit} of +2 is also issued whenever a patient is moved from the queue to a bed, ecouraging the agent to keep beds occupied rather than idling.

There is also a per step wait penalty r_{wait} that penalizes the agent for each patient sitting in the waiting room. To encode priority, the penalty is weighted by ESI level to incentivize high acuity patients being seen.

$$r_{\text{wait}} = \sum_{i \in \text{queue}} w_{\text{ESI}(i)}, \qquad w = \{5.0,\ 3.0,\ 1.5,\ 0.75,\ 0.25\} \text{ for ESI 1–5}$$

The guideline violation penalty $r_{\text{guideline}}$ is applied when the agent does not adhere to some basic medical policies. For instance, ESI 1 patients should be seen immediately (no longer than 1 step) and ESI 2 patients should be seen within 20 minutes (4 steps). If these strict guidelines are violated an extra per step penalty is applied of -10 for ESI 1 and -5 for ESI 2. This once again incentivizes critical patients being seen with priority.

The left without being seen (LWBS) penalty r_{LWBS} is applied when a patient waits 48 steps (4 hours) without being admitted. This is meant to simulate patients leaving the ED when wait times are too long since 4 hours is a rough benchmark for some EDs. This is also scaled by acuity: -50, -35, -20, -15, -15 for ESI levels 1 through 5.

4 Agents

4.1 Baseline Agents

To establish some benchmarks to compare to the neural network I implemented three baseline agents. The first is a random agent that selects a random action every step. The second is a first come first serve (FCFS) agent that always selects the patient with the longest wait time, ignoring acuity. The final is the acuity first agent which always selects the patient with the most severe ESI level and then by wait time in the event of a tie.

4.2 Deep Q-Network

The RL agent is implemented as a Deep Q-Network (DQN) with the core idea being to approximate the action-value function Q(s, a). This estimates the expected cumulative reward of taking action "a" in state "s" and following the optimal policy using a neural network. Instead of maintaining a giant lookup table over all states the network learns a compressed representation that generalizes across similar states.

The agent was expanded beyond the most simple DQN formulation to incorporate Double DQN, experience replay buffer, a frozen target network, and Huber loss. Each of these design decisions is explained below.

4.2.1 Q-Network Architecture

The Q-network takes the 65 dimensional normalized observation vector as input and outputs one Q-value for each of the 21 possible actions. A higher Q-value for an action means the network predicts that action will lead to greater long term reward from the current state.

The architecture I chose is a simple two hidden layer multilayer perceptron (MLP) implemented using Tensorflow:

$$\text{Input } (65) \xrightarrow{\text{Dense + ReLU}} 128 \xrightarrow{\text{Dense + ReLU}} 128 \xrightarrow{\text{Dense}} 21 \text{ outputs}$$

Standard ReLU activations are used in the hidden layers. Of note is not having an activation function applied to the output layer: Q-values are unbounded and can be positive or negative, so squashing them with sigmoid or softmax would be incorrect.

4.2.2 Observation Normalization

The observation vector contains features on very different scales: ESI levels range from 1-5, wait times from 0-48 steps, and treatment durations can exceed 250 steps. Without scaling the inputs the neural network gradient would be dominated by the larger scaled features making training unstable and slower. Therefore, all features were normalized within [0,1] before being passed to the network. The exception to this is the time of day which normalizes to [-1, 1] due to the sine/cosine encoding

4.2.3 Experience Replay Buffer

A key challenge in RL is that consecutive transitions (s_t, a_t, r_t, s_t+1}) are highly correlated as each state leads directly to the next. Training a neural network on correlated data violates independence and causes unstable updates.

The solution to this is storing all transitions in a **replay buffer**, a fixed size queue that holds the agent's past experiences. At each training step, a random batch of 64 transitions is sampled uniformly from the buffer breaking the step to step correlation. This exposes the network to a diverse range of situation throughout the episode.

My buffer can hold 50,000 transitions, which corresponds to roughly 170 full episodes. Training does not begin until at least 1,000 transitions have been collected ensuring that the first batches are varied enough. Each stored transition is a tuple of current state (s_t), action (a_t), reward (r_t), next state (s_{t+1}), and "done".

$$(s_t,\ a_t,\ r_t,\ s_{t+1},\ \text{done})$$

The "done" is a boolean flag indicating whether the episode ended. This is needed to zero out future rewards for terminal transitions.

4.2.4 Double DQN

Following the Double DQN implementation, two identical copies of the neural network architecture are maintained simultaneously: the **online network**, which is trained every step, and the **target network**, which has frozen weights and is periodically updated (every 500 gradient steps) and used only for computing training targets. This creates a far more stable target than training with a single network. Without this, as the network updates so will the training targets leading to a chasing your own tail situation and unstable training.

Standard DQN computes the training target as:

$$y_t = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a')$$

Theta-minus is the target network weights. The problem is that taking the max over all Q-values often overestimates the true action values in a noisy environment such as this. With 21 possible actions, there are 21 chances to select an over-inflated Q-value.

Double DQN separates action selection from action evaluation to remove this bias. The online network selects the best next action, but the target network evaluates that action:

$$y_t = r_t + \gamma\, Q_{\theta^-}\big(s_{t+1},\ \arg\max_{a'} Q_\theta(s_{t+1}, a')\big)$$

In my code this implemented as two forward passes on the next state instead of one. The online network picks the best action index based on Q-value, and the target network then uses this action index to calculate the Q-value. This way, the action is chosen by the online network, but the Q-value by the target network. This removes bias and produces more accurate training targets.

4.2.5 Loss Function

The network is trained to minimize the difference between its predicted Q-value and the Double DQN target. Rather than mean squared error (MSE), Huber loss is used.

Huber loss behaves similar to MSE for small errors and like mean absolute error (MAE) for large ones. Since per step rewards in this environment can be as large as -50 or more and compound over a 288 step episode squaring errors with MSE results in enormous gradients. This is not ideal as it can destabalize training. Huber loss clips the magnitude of this for outlier errors while still providing smooth loss near 0.

The gradients are computed with TensorFlow's GradientTape and applied with the Adam optimizer with a learning rate of 1e-3.

4.2.6 Epsilon-Greedy Exploration

The agent uses an epsilon-greedy policy to balance exploration and exploitation. At each step, it selects a random action with probability epsilon and otherwise selects the action with the largest predicted Q-value from the online network:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg\max_a Q_\theta(s_t, a) & \text{otherwise} \end{cases}$$

Epsilon is initialized to 1.0 (fully random) and decayed at the end of each episode as the agent grows more confident:

$$\epsilon \leftarrow \max\left(0.05,\ \epsilon - \frac{1.0 - 0.05}{500}\right)$$

So, as epsilon reaches its floor of 0.05 after 500 episodes the agent explores less (random actions) and instead exploits learned strategy (max Q-value). The minimum is set to 0.05 rather than 0 because the environment is stochastic meaning patient arrivals and acuity levels are random so some exploration is beneficial. During actual evaluation epsilon is set to 0.0 so the agent can act on the learned policy without noise from random actions.

4.2.7 Training Procedure

The agent was trained for 1,000 episodes. Each episode corresponds to a full 24 hour simulation (288 steps). The training loop at each step proceeds as follows:

1. The agent observes the normalized state s_t and selects an action using the epsilon-greedy approach

2. The environment processes the action and returns reward r_t and next state s_t+1}.
3. The transition is stored in the replay buffer.
4. If the buffer contains at least 1,000 transitions, a batch of 64 is sampled and a Double DQN gradient update is executed on the online network.
5. Every 500 gradient steps, the target network weights are copied from the online network.
6. At the end of each episode, the epsilon decays.

After training, the model weights are saved to a file and is evaluated with epsilon fixed at 0.0

5 Results

5.1 Evaluation Setup

All four agents: Random, FCFS, Acuity First, and the trained DQN were evaluated over 100 independent 24-hour episodes on the same environment used for training. Each episode runs for 288 steps (5 minutes per step) starting at 8:00 AM. During evaluation the DQN's epsilon is fixed at 0.0 so the policy is fully greedy with no random exploration. The baseline agents are deterministic and do not require adjustments. Metrics are averaged across all 100 episodes.

The three baselines represent simple strategies. The **Random agent** selects actions at random for each step. The **First Come First Serve (FCFS) agent** always admits the patient who has been waiting the longest, ignoring acuity entirely. The **Acuity First agent** always admits the highest acuity patient available, breaking ties by wait time. These baselines allow me to meaningfully evaluate the DQN's performance through comparison.

5.2 Summary Results

Table 3 summarizes the four agents across the most relevant metrics, averaged over 100 episodes. ESI 1 and ESI 2 max wait times are reported in steps (1 step = 5 minutes).

Table 3: Agent Comparison

Agent	Total Reward	Mean Discharges	ESI 1 Max Wait (steps)	ESI 2 Max Wait (steps)
Random	-16,278	85.6	13.91	29.38
FCFS	-16,479	95.8	12.93	24.62
Acuity First	-7,775	81.9	0.66	11.89
DQN	**-7,364**	84.9	4.45	12.21

The Random and FCFS agents performed very poorly. FCFS achieves the highest discharge count (95.8) by keeping beds constantly filled, but it pays a severe penalty because it ignores acuity entirely. ESI 1 and ESI 2 patients wait an average of nearly 65 and 123 minutes respectively far beyond clinical guidelines accumulating large wait penalties. These two agents were by far the worst and show that high throughput is insufficient, you must place a priority on acuity as well to score well with the reward function.

Acuity First addresses this directly by always treating the most critical patient first. It has amazing max wait times as shown by ESI 1 patients waiting on average just 0.66 steps and ESI 2 patients 11.89 steps. This reduces the acuity weighted step penalties that are most important in the reward function. The final total reward was -7,775 which is about half of the FCFS agent. The tradeoff is that lower acuity patients are not prioritized at all which leads to a higher LWBS penalty and fewer total discharges.

The DQN achieves the best total reward of all four agents at -7,364. This is a small but noticeable improvement over Acuity First considering it was averaged over 100 runs. It manages ESI 1 and ESI 2 wait times of 4.45 and 12.21 which are worse than pure Acuity First but substantially better than FCFS and Random. The DQN appears to have learned a mixed strategy of broadly prioritizing high acuity patients but also responds to queue pressure and bed availability in ways a naive approach cannot.

5.3 Training Curve

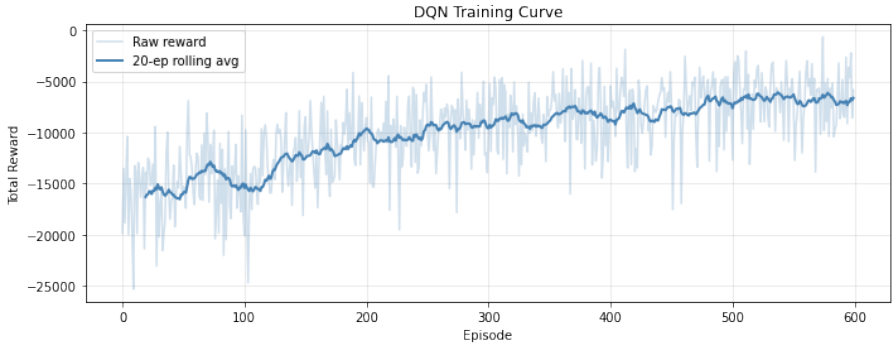

Figure 1: DQN training curve over 500 episodes. Per-episode reward (light) and 20-episode rolling average (bold).

Figure 1 shows the total reward per episode over the 500 training episodes, highlighted with a 20 episode rolling average. There is a clear trend of steady improvement since the agent begins with near random performance and converges towards a relatively stable policy as epsilon decays.

However, the raw episode rewards show substantial variance throughout training even in later episodes where the policy has mostly converged. Individual runs can differ dramatically as a direct conswquence of the environments randomness. Patient arrivals are sampled from a Poisson distribution,

ESI levels are drawn from a probability distribution, and treatment durations are sampled from a log-normal distribution. A single episode with unusually more high-acuity patients will produce a much worse reward than an episode where arrivals are sparse or well spaced. This is not something the agent has control over so it has to learn to manage a broad range of situations.

This variance also makes agent comparison to Acuity First more ambiguous. Different training runs can yield models that perform differently when evaluated. Below are two cumulative reward plots between two different training runs. As you can see in Figure 2 the DQN and Acuity First are quite close.

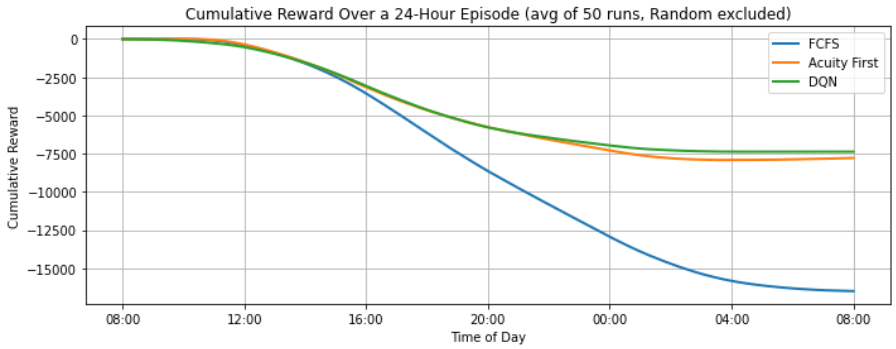

Figure 2: Cumulative reward over a 24-hour episode (avg. of 50 runs). DQN and Acuity First track closely

However, in Figure 3 you can see that a DQN from a different training run performed much stronger against Acuity First. This again demonstrates the variance you can get in your models as a reflection of the environment's unpredictability.

5.4 Wait Times by ESI Level

Figure 4 breaks down wait times by acuity level and shows the key limitation of the DQN relative to Acuity First. For ESI 1 and ESI 2 patients, Acuity First is the clear winner. Its hard rule of always

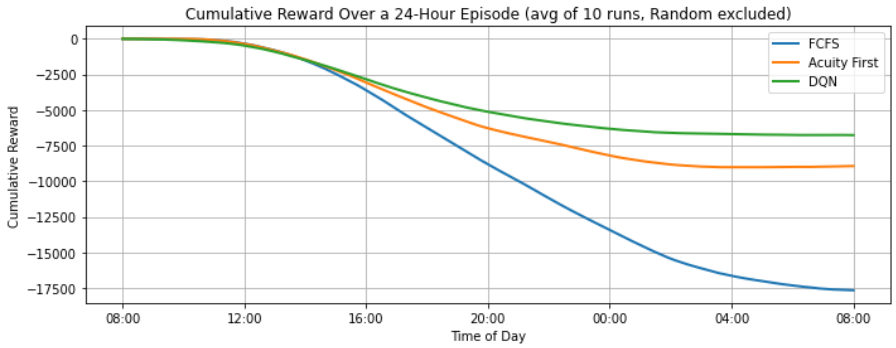

Figure 3: Cumulative reward over a 24-hour episode (avg. of 50 runs, 10 is typo). DQN and Acuity First track closely

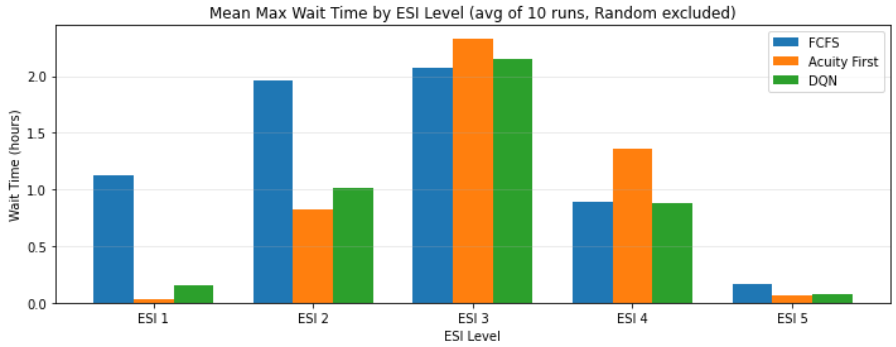

Figure 4: Mean maximum wait time by ESI level (avg. of 100 runs, Random excluded). Lower is better.

prioritizing the sickest patient available leads to the lowest critical wait times of any agent. The DQN's ESI 1 and ESI 2 wait times are longer respectively compared to Acuity First, meaning the DQN occasionally allows high acuity patients to wait when it decides to admit someone else. This suggests the agent has not fully learned the vital guideline penalties for delaying critical care which is a notable failure. Due to the stochastic and unpredictable nature of the environment it may have been harder for the DQN to learn this behavior as well as expected. I also believe that the reward system was very focused on high acuity patients which made Acuity First a difficult target to beat as it always will admit these patients as fast as possible. Perhaps by tweaking rewards to be more balanced the model could definitively outperform Acuity First.

5.5 Waiting Room Queue Length

The queue length plot reveals where the DQN makes up ground on Acuity First. Despite performing slightly worse on critical wait times, the DQN consistently maintains a smaller waiting room queue across the entire 24 hour period. Acuity First, by always choosing the highest acuity patient, tends to leave lower acuity patients stranded in the queue for extended periods. This causes the queue to grow longer and LWBS penalties to accumulate among ESI 3-5 patients which is a pattern the DQN avoids.

The DQN also appears to have learned to balance bed utilization. It keeps beds occupied with the right patients at the right time rather than rigidly following an ordering like Acuity First. This directly leads to more discharges per episode (Table 3, 84.9 vs. 81.9) and a lower queue backlog throughout the day. The reward gained from this throughput optimization and reduced lower acuity LWBS is what accounts for the DQN's edge in total reward over Acuity First, despite being worse on the wait time metric alone.

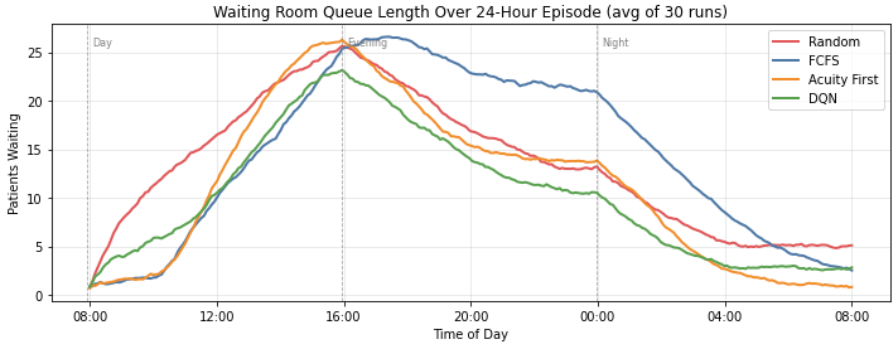

Figure 5: Mean waiting room queue length over a 24-hour episode (avg. of 100 runs)

6 Conclusion

This project demonstrated that a Deep Q-Network can learn a reasonable policy for managing emergency department boarding which is a task that requires balancing clinical urgency, patient throughput, and bed efficiency simultaneously under unpredictable conditions. A simulation environment was built from data extracted from the MIMIC-IV-ED dataset, including realistic arrival rates, ESI acuity distributions, and treatment duration distributions. Three baseline agents were implemented and evaluated, and a Double DQN agent was trained against them over 1,000 episodes.

The central finding is that the DQN achieves the best total reward of all four agents, outperforming even the surprisingly strong Acuity First. The two agents are closely matched, but it is a consistent improvement across 100 evaluation episodes. The DQN has learned to adapt its admissions strategy to the current state of the waiting room and bed availability rather than following a fixed ordering. It pays for this with slightly longer ESI 1/2 wait times, but offsets the cost through better throughput and waiting room queue management.

The key takeaway is that simple heuristics like Acuity First, while strong, cannot anticipate the implications current actions have on the future state of the emergency department. Reinforcement learning can understand these kinds of tradeoffs over time through trial and error. The gap between the DQN and Acuity First would likely widen in more complex environments where more variables interact making RL an increasingly innovative approach as the problem scales.

6.1 Future Directions

One of the most immediate changes that might strengthen this project is reward function refinement. Training on a personal machine limited how many configurations can be explored before the deadline, and the current reward weights were not fully optimized. Tuning the penalties for ESI 1/2 and the LWBS thresholds more carefully could push the agent toward better performance with critical patients without sacrificing throughput. Reducing episode to episode reward variance would also make training more stable and results more interpretable.

A second improvement could be building a more realistic environment. Real emergency departments involve far more variables than what I included such as staffing levels, specialized treatment rooms, equipment availability, mass casualty incidents, and department transfers. Adding this complexity would make the simulation more applicable to real decision making in an ED setting. It would also likely favor RL more strongly since the harder the problem is the more a learned policy can gain over hand coded naive approaches.

7 Disclosure

I used the AI model ChatGPT to help format some of the Latex and figures.

References

[1] Wei, J., Ouyang, C., Wickramanayake, B., He, Z., Perera, K., & Moreira, C. (2025). Curation and Analysis of MIMICEL - An Event Log for MIMIC-IV Emergency Department. *arXiv preprint arXiv:2505.19389*.

[2] Delos Reyes, R., Capurro, D., & Geard, N. (2024). Modelling patient trajectories in emergency department simulations using retrospective patient cohorts. *Computers in Biology and Medicine*, 182, 109147.

[3] Lee, S., & Lee, Y. H. (2020). Improving emergency department efficiency by patient scheduling using deep reinforcement learning. *Healthcare*, 8(2), 77. MDPI.

[4] Li, Y., Mao, C., Huang, K., Wang, H., Yu, Z., Wang, M., & Luo, Y. (2024). Deep Reinforcement Learning for Efficient and Fair Allocation of Healthcare Resources. *arXiv preprint arXiv:2309.08560*.